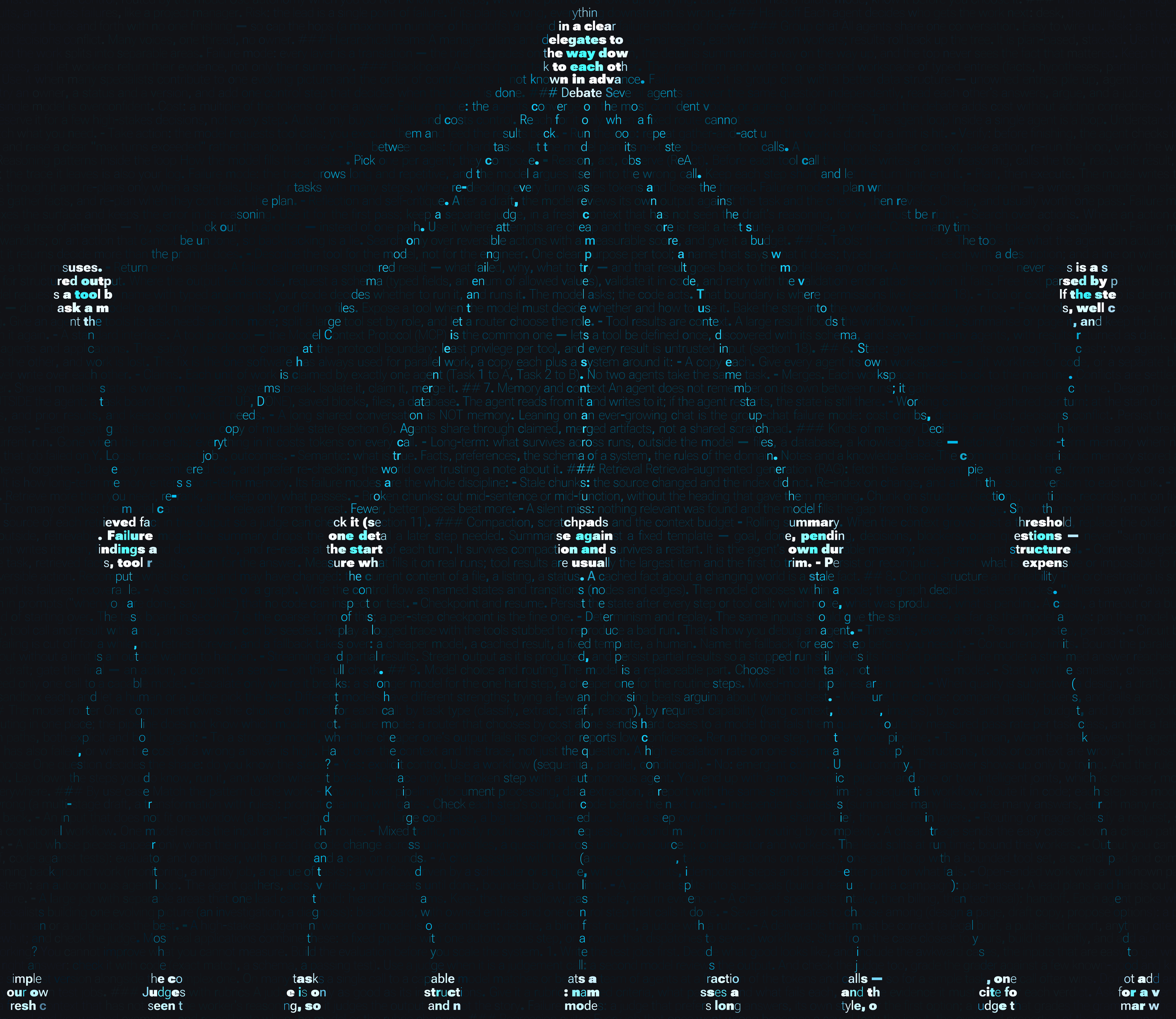
Agentic Engineering: Building Multi-Agent Systems A practical, build-ready guide to orchestrating AI agents, written for a machine to read and apply. It is general: it names the patterns for building agents and, for each kind of application, shows which pattern to reach for, how to hold state and memory, and how to choose the model. The claim throughout: these systems are not magic. They are ordinary software built with AI components. Design them the way you design software, and add intelligence only where it earns its place. ## 1. The two questions Every multi-agent design is the answer to two questions: 1. Who controls the flow? Who decides what happens next. 2. How do agents share state? How they share what they know. Each answer sits on a spectrum. At one end is EXPLICIT CONTROL: you decide everything up front, in code. At the other is EMERGENT CONTROL: the model decides at run time. Most real systems live in between, an explicit skeleton with intelligence at the joints. Name where each part of your system sits before you build it. The sections follow the build: the shapes of control (2, 3), the loop inside one agent (4), its tools (5), state and memory (6, 7), the control structure that makes it reliable (8), the model (9), how to choose (10), how to measure and improve (11, 12, 13), a worked system and a procedure (14, 15), prompts and rules (16, 17), and the authority an agent is given (18). ## 2. Workflow patterns: explicit control, routed in code Use a workflow when you already know the steps. The route is fixed in code; the model only fills in each step. Three basic shapes cover almost everything; the five that follow are the compositions of them you will reach for most. Two of the five — orchestrator-workers and evaluator-optimiser — let the model set the subtasks or decide when to stop, so they sit nearest the middle of the control spectrum (section 1); the skeleton around them is still explicit. ### Sequential One step after another. Use it when the steps are known and ordered: draft, then check, then publish. ### Parallel Fan out, then bring the results back. Use it for parts that



agent gathers sources, does the work, and runs its own checks for accuracy, structure, and writing. Then a judge, a second agent seeing the work for the first time, reads the report. After a few fix cycles it is handed back to the human. During testing, plant a fault on purpose (a quote changed to claim more than it says) and confirm the judge catches it. - Fan-out: split the work into several isolated sandboxes, each running a different model (for example three frontier models from different providers) building the same deliverable. A copy each, no shared state. - Human at the end: the person picks the best of the several outputs. That is the only routine human step; escalation (section 9) and the gate before any irreversible action (section 18) can still bring a human in mid-run — a send or a publish passes through that gate, not straight out. - Everything is logged: a live event stream and an explicit per-step status, so you can see what each agent did and why. The board holds the checkpoint; the stream holds the trace. ## 15. Build procedure Do this, in order: 1. Define good: write the test jobs, including the awkward and the adversarial cases. 2. Start with one call to the smallest model that could work. Measure quality, tokens, and calls. 3. If one call is enough, ship it. It often is. 4. If not, draw the workflow (sequential, parallel, conditional, or one of their compositions: a gated chain, routing by complexity, orchestrator and workers, evaluator and optimiser, map-reduce) and write the route in code as a graph with named states. 5. Find the one step that cannot be pre-scripted. Make only that step an autonomous agent, pick its pattern (plan-based, handoff, group chat, hierarchical, blackboard, debate) with its failure-mode guard, and pick its reasoning shape (section 4). 6. Design the tools: one purpose each, typed schemas, errors as data, deterministic logic in code (section 5). 7. Design the memory: keep durable state outside the agents (a board, files), and give each agent its own working copy; add claims and merges for shared artifacts; decide what is retrieved, what is summarised, what is persisted and